DP

Answer Format

Subproblem Definition
Simply follow this format:
T(i) = <reword problem> in <input subset> <whether a_i must be used>

Honestly just stick with using T(*) to define all problems. It makes the answering process much more consistent.

Recurrence Relation
Checklist:

1. Include the base case
2. Include the range (i.e., where to start and where to stop)
3. The inputs are all indexed from 1. In my opinion it makes everything easier to think about.
    - However, our own tables can be indexed from anywhere. Usually the base cases start at T(0). T(-1) is also allowed, if you really fancy that.
4. Do not reference unset values
5. Do not self-reference. T(i) = T(i) is wrong.
6. Do not set conflicting values.
    - It is okay to assign the *same value multiple times*, as long as the values are not conflicting.
    - For example, suppose I wish to set my base cases in a 2D table where the first row/column and the diagonal are all set to zero.
        - For the diagonal, I would go about setting it as `T(i, j) = 0 if i == j`
        - For the initial row/column, I would set them as `T(i, 0) = 0` and `T(0, j) = 0`
        - Notice how if `i == j == 0`, the top-left cell `T(0, 0)` will be set multiple times. Again, this is perfectly fine as long as the same value is being set.

Implementation Analysis
Checklist:

1. State the number of subproblems in big-Oh (this is the lower bound for overall time complexity)
2. State the runtime to fill the table T . Note that this is different from the point above, though this will be clearer when looking at DP examples below.
3. State how/where the final return value is extracted.
4. State the runtime to extract this return value in big-O.

Strategies

DP Identity

First, there are three main categories of DP that this class tries to teach. Try to figure out which category is relevant, start with some generic approach, then fit specific details of the problem being tackled. These three categories are not given consistent names, but here they are:

1. Sequences (LIS, LCS)
2. Windowing (CMM, Optimal BST)
3. Bag (Knapsack, Making Change)

I don't like the names being used, so here are mine:

1. Sequences; self-explanatory
2. Segments (`A[i, ..., j]`). The segments are described by the start i and the end j. The best way I can think about this is by drawing upon the CMM example and how subtrees are defined.
3. Subsets (`{A[i], ..., A[j]}`). The subsets are described by items i to j, though not all of them have to be part of the solution. Only a subset is required.

Sequences

Longest Increasing Subsequence (LIS)

The problem is likely LIS if it:

- Only has a single input array
- Involves comparing the input to itself
- Involves looking back at one element at a time (i.e., not a window/segment or a bag/subset).

This is normally a 1D table, thus there are O(n) subproblems.

There are two variations:

1. With O(n) lookback.
    - If the subproblem needs to check all previous elements OR
    - If the subproblem needs to check all previous elements with some additional requirements (e.g., less than threshold, more than current value, etc.)
    - This results in a total runtime of O(n^2).

2. With O(1) lookback
    - If the subproblem needs to check only one element back. In the same vein, if the subproblem *carries* the max/sum/count/etc. forward OR
    - If the subproblem needs to check a constant number of elements back.
    - This results in a total runtime of O(n)

Longest Common Subsequence (LCS)

The problem is likely LCS if it:

- Is comparing between two arrays
- Is looking for something in common

This is normally a 2D table.

There are two variations

1. Substring
    - If the subproblem needs the matches to be consecutive.
    - Add 1 (or whatever value) if the comparison succeeds: `T(i, j) = 1 + T(i-1, j-1)`
    - Reset to 0 whenever a comparison fails
    - The solution is the maximum of T(*)

2. Subsequence
    - If the subproblem allows the comparison to skip mismatches
    - Add 1 (or whatever value) if the comparison succeeds: `T(i, j) = 1 + T(i-1, j-1)`
    - Take the max of either side if the comparison fails: `T(i, j) = max{ T(i, j-1), T(i-1, j) }`
    - The final result is the final cell of the table `T(n, m)` or `T(n, n)`

Runtime is O(n*m) or O(n^2) . This is true for both variations on LCS.

Edit Distance

The problem is likely edit distance if:

- It is comparing between two arrays
- Is looking to minimise differences
- Assigns some penalty to a difference
- Assigns some points to a match
- Is aligning two arrays

Note that this problem is very similar to LCS, but is different enough to be considered a 3rd variation on the Sequences class.

This is normally a 2D table. In general:

```
T(i, j) = min/max {
        T(i-1, j-1) + f(i, j),
        T(i-1, j) + f(i, j),
        T(i, j-1) + f(i, j)
}
```

where f(i, j) is some function that determines the value/penalty/etc. of aligning i and j .

Concretely, consider the "standard" edit distance problem, where penalties are only associated with deletions, substitutions, or insertions. Let the two strings to compare be x and y . Then, define f(i, j) = 0 if x[i]==y[j] else 1. I'm assuming the format is exactly the same if alignments are rewarded; I've not encountered variations of edit distance thus far.

The final result is the final cell of the table T(n, m) or T(n, n). Runtime is O(n*m) or O(n^2).

Subsets (Knapsack)

The problem is likely knapsack (or reduces to knapsack) if it:

- Needs to check if something can fit or add up to (a variant of this is the subset sum problem)
- Needs a maximum value for a specific budget
- Needs to find the minimum budget needed for a specific value

There are two variations

1. Limited Items
    - This is normally a 2D table, with O(n*B) subproblems
    - For each increment of the items and for each increment of the budget:
        - If the item can fit `w[i] <= b`
            - Then `T(i, b) = max{ T(i-1, b), T(i-1, b-w[i]) + v[i] }`
            - This is checking if item i is worth adding -- is it better to ignore item i and keep the current budget, or is it better to decrease the available budget and add item i ?
        - If the item cannot fit:
            - Then `T(i, b) = T(i-1, b)`
            - This is simply ignoring item i.
    - Runtime is O(nB)

2. Unlimited items
    - The main difference here is that we don't check i-1 because we don't care if i was considered or not.
    - This is normally a 1D table, though it can be a 2D table (see lectures). The usage of either depends on your own comfort level; both will be accepted.
        - There are O(B) or O(nB) subproblems for the 1D and 1D table respectively.
    - For each increment of the items and for each increment of the budget:
        - If the item can fit `w[i] <= b`
            - `T(b) = max{ T(b-w[i]) + v[i] : w[i] <= b, 1 <= i <= n.`
              `= 0 otherwise`
            - Again, this is checking if this instance of item i is worth adding. We remove enough room to include this instance of item i.
        - If the item cannot fit:

        - Don’t do anything. It is as if the item was never considered.
    - Runtime is O(nB).

The intuition seems to be "removing items currently in the bag and trying to fit in other items, see if that works better". Convince yourself that the items don't need to be sorted.

Segments (Windowing)

The problem is likely involves windowing if it:

- Needs a windowing check
- Is asking for the most efficient way to split an array
- Is asking if the array can be split in some way to do something.

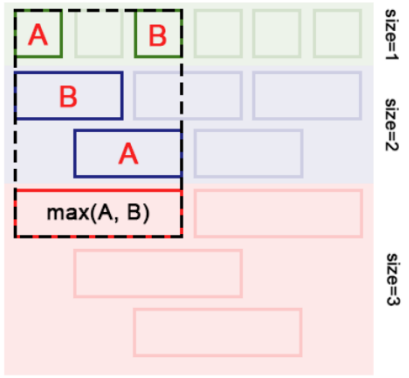
This is normally (there are exceptions) a 2D table, with O(n^2) subproblems.

There are two variations:

1. Windowing only
    - Start with a window of size of `1->n` . Set `i = 1 -> n - window - 1` , set `j = i + window + 1`. This is just arithmetic for indexing and ensuring that the index does not overflow.
    - Calculate T(i, j) using smaller windows between i and j.
    - Runtime is O(n^2).

2. Windowing and breaking at midpoint
    - Start with a window size of 1->n . Set `i = 1 -> n - window - 1`, set `j = i + window + 1`.
    - We also need a `k = i+1 -> j-1`. This is a breakpoint that maximises T(i, j) . There is no need to track k , as only T(i, j) is required.
    - Runtime is O(n^3)

The illustration below shows how larger subproblems build upon smaller subproblems.



Notice how if we need to consider the optimal breakpoint k , then we need to consider all possible combinations (illustrated above as taking the maximum of A and B). This scanning for optimal k results in O(n^3)

Workflow

As tackling a DP problem always starts with rote subproblem definition, it might be helpful to consider the form of table T that will be used. Will it be a 1D array or 2D/3D matrix?

- If the value/cost/profit function involves only one input, then it is likely that a 1D array is being used (e.g., LIS). If two inputs are required (e.g., matrix multiplication or LCS), then it's likely a 2D matrix.
- Alternatively, determine how many dimensions are required based on the number of parameters required to describe a subproblem (I like this interpretation a lot right now). For example, a Knapsack state requires knowledge of the current capacity and value, thus a 2D matrix is warranted.

DP is inherently "brute force". Don't be afraid to compute literally everything. This is especially clear in Windowing/Segment-type problems, where all possible windows are computed and stored.

A table entry might not necessarily be the best answer for that particular subproblem. Take for example Longest Common Substring, where the final return value has to involve scanning the entire matrix for max{T(*)} . Another example would be the H otels problem (DPV 6.2).

- I think there is something to be said about retaining "suboptimal" solutions in order to achieve the global optimum. I admit that this line of thinking is a bit weird with Longest Common Substring where 0s are entered into T if the substring condition is violated.

A few examples/scenarios/test cases are **absolutely mandatory**. I don't think I've gotten to a recurrence relation without first testing out ideas on small examples.

- Then, should I be filling in T based strictly on my subproblem definition (which might be insufficient), or with what I think is the correct answer?
- I think either works, but my current preference seems to be filling it with the correct answer. If I find that I cannot inform the correct answer with answers from previous subproblems, then my subproblem definition needs to be strengthened.

Miscellaneous

1. Subsequences are non-contiguous. Substrings are contiguous.

2. Note that there are no limits on space complexity.

3. If struggling with Boolean tables, try switching to using integer representations (0 and 1). It is much easier to think about maximums/minimums and arithmetic operations instead of logical operations.

DC

Note: I lost a lot of points in the DC portion of Exam 1, because I spent too much time worrying about DP. Don't let this segment slide out of concern; it will hit like a truck.

Answer Format

Algorithm

Checklist:

1. No pseudocode
2. Include the base case (where does the recursion end?)
3. If a black box is used, explain what part will be modified if needed.
4. Too much detail is better than too little.
5. Bullets are allowed but not preferred. If possible, use prose.
    a. Subbullets are discouraged
6. Avoid variable assignments or function name calls. Use words instead.

Justification of Correctness

Checklist:

1. Why did you pick the black box? For example, "binary search was picked because the array was already sorted…"
2. Why did you make modifications to the black box?
3. Why does the final step in your recursion solve the problem?

Runtime Analysis
It is not enough to simply state the runtime (like DP). Justification is required.

If a black box is used, first state the original runtime of the unmodified algorithm. Then, state the changes made and how that affects the runtime. Otherwise, if a new algorithm is proposed, then go into detail when performing runtime analysis (this operation takes O(n), and this takes O(n^2), and so forth).

The Master Theorem can be used here as well. The math does not need to be shown, but the values a , b and d have to be justified. For example, "I am dividing my problem into two subproblems of size n/2 ".

DC Algorithms

A DC algorithm should follow the general approach:

1. Break input into several parts (divide)
2. Solve each part independently
3. Eliminate, merge, or cancel our (conquer)

In DP, we "brute forced" every possible subproblem in order to answer a larger problem. However, we are actually trying to do as little work as possible in DC. This is an important feature of the conquer step.

- In binary search, we are *eliminating* half of the array at every step, since the answer is not going to be in the eliminated half anyway.
- In merge sort, we compare smaller pairs (instead of comparing everything) before merging the solution up to the top.
- In fast integer multiplication, we noticed that some operations are duplicated and can therefore be *cancelled.*

It is immediately apparent that coming up with a solution to a problem *without* divide and conquer is actually pretty trivial. Finding out if a number exists in an array is trivial with a linear scan. The focus of divide and conquer is to find efficient/optimal ways to solve the problem.

If DP is solving problems by solving smaller subproblems, DC is solving problems by avoiding smaller subproblems.

Classic Algorithms

Algorithm	**Runtime**
Binary search	O(log n)
Merge sort	O(n log n)
Fast select	O(n)
Fast multiply	O(n^{log_2^3})
FFT	O(n log n)

Note that while Quicksort and Quickselect are mentioned in the lectures, these are *not optimal* and serve as foundations for improvement (Merge sort and Fast select). Do not use the quick* variants.

Strategies

Black Box Choice

Always start with the brute-force algorithm. For example, finding an element in an array simply involves a linear scan, which takes time O(N). The answer should be at least faster than the brute-force algorithm (in this case, binary search takes O(log N)). Correctness is important, but it will not help if the algorithm given is equivalent or worse than brute-forcing.

It seems that the strategies are tied to the input problem being sorted or unsorted.

1. If the input is sorted, then consider binary search
    - Consider modifying the target. Instead of looking for some element x , maybe the problem warrants looking at something else.
    - Consider modifying the branching logic. Instead of splitting each problem into n/2 , maybe try n/3 , or perhaps even an unequal split.
    - Or any time a linear search is required on a sorted input, use binary search. This can be a component of a larger solution.

2. If the input is unsorted, then there are a few strategies to consider.
    - It is possible to sort first O(n log n) , but this is not always the better option. Be careful with this.
    - Consider modifying the merge logic to merge sort.
    - Consider modifying the partitioning logic to fast select. Instead of grouping into three buckets based on the pivot, perhaps group into five based on some range of values from the pivot.

Other DC strategies to consider include:

- Fast Multiply or FFT for math-related problems
- Tournament algorithm
    - The idea here is to pair and eliminate as we move up. I frame this to be under Merge Sort but I guess Joves thinks that it is different enough.

I think there is merit in considering if I can "verify" after obtaining candidate solutions. For example, the assigned problems (e.g., majority element and k towers) seem to use divide-and-conquer to obtain candidates, then proceed to verify the candidates in some linear/log time. In short, there is really no need to use DC black-boxes to solve the problem outright.

Master Theorem

Of course, know the Master Theorem. Know how to establish `T(n) = aT(n/b) + O(n^d)`

- a is the number of recursive calls (i.e., number of subproblems)
- b is the size of partition in each recursive call (i.e., subproblem size)
- n^d is the effort at each recursion (i.e., work at the conquer step)

Then, the total runtime is simply pattern matching if the Master Theorem permits. If not, you will likely have to use substitution, or just draw the recursion tree out if possible.

- `O(n^d) if logb (a) < d`
- `O(n^d log n) if logb (a) = d`
- `O(n^{logb (a)} if logb (a) > d`

Common Mistakes

Obviously do not give a brute force solution (very large penalty) and try to avoid suboptimal solutions (some penalties may apply). The “some” word here is putting in a lot of work. Try to avoid suboptimal solutions like the plague -- penalties are extremely hefty.

Algorithm

- Missing or incorrect base case. The algorithm needs to know when to stop.
- Recurrence splitting/branching lacks details or is incorrect.

Justification of Correctness

- Does not explain why the algorithm branches to the left/right. For binary search in particular, be explicit about why a certain branch is discarded -- the value can never be found in the left/right.
- Does not mention that the array being sorted allows for the use of binary search. For some reason this is penalised, even though it should be obvious. Just state explicitly that the array being sorted allows for binary search.

Runtime Analysis

- Correct but ignores black box modification analysis. Regardless of how trivial the modification is, just state how it will or will not impact runtime.